E-Commerce Price Tracker (PricePulse)

End-to-End System Manual: Architecture, 16 Phases, Troubleshooting & Codebase

1. Executive Summary & Polyglot Persistence Architecture

PricePulse is an automated web scraping, time-series price monitoring, and analytics engine. It combines **relational database modeling** (SQLite/SQLAlchemy) for structured product entities and price checkpoints with **document-based NoSQL storage** (MongoDB/PyMongo) for raw DOM snapshots and request payloads. The system features automated background tasks via APScheduler, Matplotlib headless chart rendering, and a Flask dashboard.

Layer	Technology	Key Responsibilities
Relational DB	SQLite / SQLAlchemy ORM	Products, categories, price history checkpoints, audit scrape logs
Document DB	MongoDB / PyMongo	Raw HTML DOM snapshots, response headers, schema-less metadata
Web Scraper	Requests / BeautifulSoup4	HTTP session pooling, custom headers, price/stock/rating parsing
Analytics Engine	Pandas / NumPy	Moving averages, standard deviations, percentiles, buy/wait heuristics
Visualization	Matplotlib (Headless Agg)	Base64 PNG rendering for historical trendlines & target thresholds
Web & Scheduling	Flask / Jinja2 / APScheduler	Dashboard UI, CSV exports, JSON REST API, recurring cron worker
Containerization	Docker & Docker Compose	Multi-container orchestration (Flask + MongoDB 7.0 + Volumes)

2. Troubleshooting Log & Error Resolutions

Issue / Error Message	Root Cause	Applied Resolution
AttributeError: SQL_DATABASE_URI	Config key mismatch with DATABASE_URL	Added fallback resolution in SQLDatabase constructor
ModuleNotFoundError: apscheduler	Package missing in virtualenv	Executed pip install apscheduler inside active venv
AttributeError: SCRAPE_INTERVAL_HOURS	Missing default in Config object	Defined SCRAPE_INTERVAL_HOURS = 6 in config.py
SyntaxError in services/scheduler.py	File redirection typo during piping	Replaced duplicate line with clean single import
Browser Bing Search Loop	Markdown URL syntax pasted in address bar	Navigated directly to plain http://127.0.0.1:5000
docker: term not recognized	Docker Desktop missing on host	Specified AMD64 Windows installer with WSL 2 backend

3. 16-Phase Execution Roadmap

Phase	Description	Primary Artifacts
Phase 1	Virtualenv, requirements, and environment setup	requirements.txt, .env, config.py
Phase 2	Relational database schema & ORM models	database/sql_database.py
Phase 3	Document NoSQL database schema & indexes	database/mongo_database.py
Phase 4	Domain OOP models and value objects	models/product.py
Phase 5	Web scraping core engine & DOM parser	scraper/base_scraper.py, product_scraper.py
Phase 6	Pandas/NumPy statistical analysis engine	analytics/price_analysis.py
Phase 7	Headless Base64 visualization generator	visualization/charts.py
Phase 8	Orchestration service & workflow pipeline	services/price_tracker.py
Phase 9	Comprehensive 22-test automated verification	tests/test_*.py
Phase 10	Synthetic 14-day time-series history seeder	seed_history.py
Phase 11	Jinja2 responsive Bootstrap 5 web templates	templates/*.html

Phase 12	Flask web server controllers & routes	app.py
Phase 13	Asynchronous APScheduler background worker	services/scheduler.py
Phase 14	CSV export engine & REST JSON API	app.py (/export/csv, /api/products)
Phase 15	Price drop alert & webhook notifications	services/notifier.py
Phase 16	Docker multi-container orchestration & PDF report	Dockerfile, docker-compose.yml

4. Automated Verification Suite Logs (22 Passed)

```
============================= test session starts ==============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\pasal\E-Commerce-price-Tracker
collected 22 items

tests/test_database.py::test_add_product_and_history PASSED              [  4%]
tests/test_mongo.py::test_insert_and_retrieve_raw_scrape PASSED          [  9%]
tests/test_models.py::test_product_creation_and_latest_price PASSED      [ 18%]
tests/test_scraper.py::test_product_scraper_parse PASSED                  [ 36%]
tests/test_analytics.py::test_summary_statistics PASSED                  [ 54%]
tests/test_analytics.py::test_recommendations PASSED                     [ 63%]
tests/test_visualization.py::test_price_history_chart_success PASSED   [ 72%]
tests/test_services.py::test_add_product_to_track PASSED                 [ 81%]
tests/test_services.py::test_get_product_details_with_analytics PASSED [ 86%]
tests/test_app.py::test_dashboard_route PASSED                           [ 95%]
tests/test_app.py::test_add_page_route PASSED                            [100%]

============================== 22 passed in 10.47s ==============================
```

5. Complete Core Codebase

config.py

```
import os
from dotenv import load_dotenv
load_dotenv()

class Config:
    SECRET_KEY = os.getenv('SECRET_KEY', 'default-key-123')
    DATABASE_URL = os.getenv('DATABASE_URL', 'sqlite:///data/ecommerce.db')
    SQL_DATABASE_URI = DATABASE_URL
    MONGO_URI = os.getenv('MONGO_URI', 'mongodb://localhost:27017/')
    MONGO_DB_NAME = os.getenv('MONGO_DB_NAME', 'ecommerce_raw_data')
    SCRAPE_INTERVAL_HOURS = int(os.getenv('SCRAPE_INTERVAL_HOURS', 6))
    REQUEST_TIMEOUT = int(os.getenv('REQUEST_TIMEOUT', 10))
    USER_AGENT = os.getenv('USER_AGENT', 'Mozilla/5.0 Chrome/122.0')
```

services/price_tracker.py (Orchestrator)

```
class PriceTrackerService:
    def __init__(self, sql_db=None, mongo_db=None, scraper=None, notifier=None):
        self.sql_db = sql_db or SQLDatabase()
        self.mongo_db = mongo_db or MongoDatabase()
        self.scraper = scraper or ProductScraper()
        self.notifier = notifier or NotificationService()

    def add_product_to_track(self, url, target_price=None, category=None):
        scraped = self.scraper.scrape_product(url)
        if not scraped: return None
        prod = self.sql_db.add_product(scraped['title'], url, target_price, category)
        rec = self.sql_db.add_price_record(prod.id, scraped['price'], scraped['in_stock'], scraped['rating'])
        self.mongo_db.insert_raw_scrape(prod.id, url, 200, scraped['raw_data'], scraped['html_snapshot'])
        if target_price and scraped['price'] <= target_price:
            self.notifier.send_price_drop_alert(prod.title, scraped['price'], target_price, url)
        return prod
```

app.py (Flask Web App & API Engine)

```
from flask import Flask, render_template, request, redirect, url_for, flash, jsonify, Response
app = Flask(__name__)

@app.route('/')
def index():
    products = sql_db.get_all_products(active_only=False)
    return render_template('index.html', products=products)

@app.route('/product/<int:product_id>/export/csv')
def export_csv(product_id):
    history = sql_db.get_price_history(product_id, limit=500)
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(['Record ID', 'Product ID', 'Price', 'Stock', 'Timestamp'])
    for r in history: writer.writerow([r.id, product_id, r.price, r.in_stock, r.scraped_at])
    return Response(output.getvalue(), mimetype='text/csv',
                    headers={'Content-Disposition': f'attachment;filename=history_{product_id}.csv'})

@app.route('/api/products/<int:product_id>')
```

```
def api_product(product_id):
    details = service.get_product_details_with_analytics(product_id)
    if not details: return jsonify({'error': 'Not found'}), 404
    details.pop('charts', None)
    return jsonify(details)
```

docker-compose.yml

```
services:
  mongodb:
    image: mongo:7.0
    ports: ['27017:27017']
    volumes: [mongo_data:/data/db]
  web:
    build: .
    ports: ['5000:5000']
    environment:
      - DATABASE_URL=sqlite:///data/ecommerce.db
      - MONGO_URI=mongodb://mongodb:27017/
      - SCRAPE_INTERVAL_HOURS=6
    volumes: [sqlite_data:/app/data]
volumes:
  mongo_data:
  sqlite_data:
```